

SDK / стек

Параметр	Значение
Название приложения	Ice Fishing
Android Gradle Plugin	9.2.1
minSdk	32
targetSdk	36
Kotlin	да, ~2.1
Web View	нет
Custom Tabs	да
Рекламные сети	нет
Аналитика	нет
Permissions	android.permission.INTERNET, com.google.android.gms.permission.AD_ID, com.ice.fishing.grenza.DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION, com.google.android.finsky.permission.BIND_GET_INSTALL_REFERRER_SERVICE, com.android.vending.CHECK_LICENSE
Libraries	Kotlin ~2.1 / kotlin-stdlib, kotlinx-coroutines 1.9.0, kotlinx-serialization, Jetpack Compose 1.11.4 / Material3 1.4.0, AndroidX (Activity 1.13.0, Browser 1.9.0 Custom Tabs, Core 1.19.0, Lifecycle 2.11.0, Navigation 2.9.8, Room 2.8.4, SQLite 2.6.2, Emoji2 1.4.0, Startup 1.2.0, ProfileInstaller 1.4.0, Window 1.5.0, SavedState 1.4.0, Hilt Navigation Compose 1.4.0), Dagger/Hilt 2.60.1, OkHttp 5.4.0, Google Play Services Ads Identifier 18.2.0 / Base / Basement / Tasks, Play Install Referrer, Pairip (com.pairip.application / licensecheck)
Подозрительные домены	eternalsiegeapplication.space
SharedPreferences	Room DB app_db: таблица gate_config (id, valueSaved, customTab, customTabLink) — кэш решения клоаки и URL оффера; таблица items — каталог «блюд» белой оболочки; SharedPreferences com.google.android.gms.signin — служебные prefs GMS
Есть ли клоака	да
Подозрительные слова	{{OFFER_URL}}, offer, customTab, customTabLink, GateConfig, openTab, tabUrl, Settled, User-Agent, anatoliaTable, WebSocket, spiceLantern, bazaarSaffron, copperPlatter, tableStory, scoreVisible, valueSaved, eternalsiegeapplication

Проверка домена: eternalsiegeapplication.space

Параметр / движок	Значение / вердикт
Домен	eternalsiegeapplication.space
VirusTotal URL	https://www.virustotal.com/gui/domain/eternalsiegeapplication.space
Детекции	0/91 (malicious=0, suspicious=0)
Security vendors' analysis	ниже построчно, как на VirusTotal
Abusix	Clean
Acronis	Clean
ADMINUSLabs	Clean
AILabs (MONITORAPP)	Clean
AlienVault	Clean
Antiy-AVL	Clean
BitDefender	Clean
Blueliv	Clean
Certego	Clean
Chong Lua Dao	Clean
CINS Army	Clean
CRDF	Clean
CTX AI	Clean
Cyble	Clean
CyRadar	Clean
desenmascara.me	Clean
Dr.Web	Clean

Параметр / движок	Значение / вердикт
EmergingThreats	Clean
Emsisoft	Clean
ESET	Clean
ESTsecurity	Clean
Forcepoint ThreatSeeker	Clean
Fortinet	Clean
G-Data	Clean
Google Safe Browsing	Clean
GreenSnow	Clean
Heimdal Security	Clean
IPsum	Clean
Juniper Networks	Clean
Kaspersky	Clean
Lionic	Clean
Malwared	Clean
MalwarePatrol	Clean
Netcraft	Clean
OpenPhish	Clean
Phishing Database	Clean
Phishtank	Clean
PREBYTES	Clean
Quick Heal	Clean
Quttera	Clean
Scantitan	Clean
SCUMWARE.org	Clean
Seclookup	Clean
Sophos	Clean
StopForumSpam	Clean
Sucuri SiteCheck	Clean
ThreatHive	Clean
URLhaus	Clean
Viettel Threat Intelligence	Clean
ViriBack	Clean
VX Vault	Clean
Webroot	Clean
Xcitium Verdict Cloud	Clean
Yandex Safebrowsing	Clean
ZeroCERT	Clean
0xSI_f33d	Unrated
alphaMountain.ai	Unrated
AlphaSOC	Unrated
ArcSight Threat Intelligence	Unrated
AutoShun	Unrated
Axur	Unrated
Bfore.Ai PreCrime	Unrated
Bkav	Unrated
ChainPatrol	Unrated
Cluster25	Unrated
CMC Threat Intelligence	Unrated
Criminal IP	Unrated
CSIS Security Group	Unrated
Cyan	Unrated
DNS8	Unrated

Параметр / движок	Значение / вердикт
Ermes	Unrated
Fortra	Unrated
GCP Abuse Intelligence	Unrated
GreyNoise	Unrated
Gridinsoft	Unrated
Guardpot	Unrated
Hunt.io Intelligence	Unrated
LevelBlue	Unrated
Lumu	Unrated
MalwareURL	Unrated
Mimecast	Unrated
PhishFort	Unrated
PrecisionSec	Unrated
SafeToOpen	Unrated
Sansec eComscan	Unrated
SecureBrain	Unrated
Snort IP sample list	Unrated
SOCRadar	Unrated
URLQuery	Unrated
VIPRE	Unrated
ZeroFox	Unrated
Куда редиректит	нет (без редиректа)
Что выводит (кратко)	{"error": "Not Found"}
Где припаркован	регистратор: NAMECHEAP INC

Как работает клоака

В начале

Снаружи Ice Fishing выглядит как спокойный каталог турецкой кухни «Welcome to Anatolia»: блюда, оценки, факты и мини-игра крестики-нолики. В Google Play так и описано, внутри APK есть полноценный Jetpack Compose интерфейс с локальной базой блюд (таблица items).

Обман в другом. При запуске приложение не сразу показывает каталог. Сначала оно тихо собирает идентификаторы устройства и установки, затем по WebSocket (постоянное сетевое соединение) спрашивает у своего сервера eternalsiegeapplication.space, можно ли этому человеку открыть «боевую» ссылку.

Модератору магазина и «неподходящему» трафику обычно оставляют безопасную белую оболочку — нативное меню с блюдами и крестиками-ноликами. «Нужному» пользователю сервер присылает URL оффера (или плейсхолдер {{OFFER_URL}}, который подменяется на адрес того же домена). Эту ссылку открывают в Custom Tabs (встроенная вкладка Chrome поверх приложения), и человек уходит на внешний лендинг. Финальный сайт в APK не зашит жёстко: адрес приезжает динамически в поле tableStory ответа сервера.

Кому что показывают

Белая страница / обычный режим. Её видит тот, кому сервер не велел открывать вкладку (флаг customTab / openTab = false) или прислал пустой URL: типично проверка Google Play, бот, эмулятор или «нецелевой» трафик по правилам на стороне сервера. Человек остаётся в нативном Compose-приложении: вкладки Table / Scored / Fun, поиск блюд, оценки, tic-tac-toe. Снаружи это выглядит как честный кулинарный каталог.

Чёрный / боевой режим. Туда попадает пользователь, для которого ответ AnatoliaTablePayload помечает ссылку как оффер (http/https или {{OFFER_URL}}) — тогда Settled(openTab=true, tabUrl=...). Приложение сохраняет решение в Room (gate_config) и при ON_RESUME открывает Custom Tabs с этим адресом. Повторные запуски, если valueSaved уже true, берут кэш из базы и снова открывают вкладку.

Как приложение решает, кого куда пустить

- 1) Advertising ID (GAID). Смотрит рекламный идентификатор Google → кладёт в поле `spiceLantern` запроса → сервер может отличать устройства и фильтровать трафик.
 - 2) Install Referrer. Спрашивает у Play Store, откуда пришла установка (кампания/источник) → поле `bazaarSaffron` → сервер видит органику vs платный/аффилиатный трафик.
 - 3) ANDROID_ID. Берёт `Settings.Secure.ANDROID_ID` → поле `copperPlatter` → стабильный идентификатор устройства для повторных проверок.
 - 4) User-Agent браузера. Берёт `WebSettings.getDefaultUserAgent` (строка как у системного браузера) и ставит её заголовком WebSocket-запроса → сервер может отсекаать ботов/необычные UA.
 - 5) Ответ сервера (мозг решения). Сервер возвращает `AnatoliaTablePayload`: среди полей — `scoreVisible` и `tableStory` (URL). Клиент считает «это оффер», если `tableStory = {{OFFER_URL}}` или начинается с `http/https` → тогда `openTab=true`. Иначе — белая игра.
 - 6) Локальный кэш `gate_config`. После ответа пишется `valueSaved` / `customTab` / `customTabLink`. При следующем старте, если `valueSaved` уже `true`, приложение может сразу взять сохранённый `customTabLink`.
- Гео/VPN/язык как отдельные явные проверки в клиентском коде не видны: фильтрация, судя по схеме, выполняется на сервере `eternalsiegeapplication.space` по присланным идентификаторам и UA.

Цепочка по шагам

- Шаг 1. Пользователь запускает Ice Fishing → открывается MainActivity на Jetpack Compose → зачем: показать «обычное» приложение и параллельно запустить гейт → система поднимает Launch-флоу.
- Шаг 2. Приложение читает Room `gate_config` (`id=0`) → зачем: понять, уже ли сохранено решение → получает `valueSaved` / `customTab` / `customTabLink` или пусто.
- Шаг 3. Параллельно собираются GAID, Install Referrer и ANDROID_ID → зачем: идентифицировать установку для сервера → три строки `spiceLantern` / `bazaarSaffron` / `copperPlatter`.
- Шаг 4. OkHttp открывает WebSocket на `https://eternalsiegeapplication.space/api/anatoliaTable` с User-Agent системного браузера → зачем: тихо спросить у «мозга», что показывать → соединение ждёт JSON-ответ до ~80 секунд.
- Шаг 5. Клиент отправляет `AnatoliaRequestFrame` (три идентификатора) → сервер на своей стороне решает белый/чёрный режим → в ответ приходит `AnatoliaTablePayload` с `tableStory`.
- Шаг 6. Если `tableStory` — `http(s)` или `{{OFFER_URL}}` → URL нормализуется (плейсхолдер → `https://eternalsiegeapplication.space/`) → состояние `Settled(openTab=true, tabUrl=...)`.
- Шаг 7. Решение пишется в `gate_config` (`valueSaved=true`, `customTab`, `customTabLink`) → зачем: не спрашивать заново при каждом старте → кэш на устройстве.
- Шаг 8. UI видит `Settled` с `openTab=true` и непустым URL → на `ON_RESUME` стартует Custom Tabs Intent → пользователь видит внешнюю страницу оффера во вкладке Chrome.
- Шаг 9. Если `openTab=false` или URL пустой → Custom Tabs не открывают → пользователь остаётся в белом каталоге блюд и tic-tac-toe.
- Шаг 10. При ошибке сети/таймауте WebSocket → в UI показывается ошибка запуска; боевой URL не открывается (в коде явного офлайн-оффера нет).

Технические уловки

Маскирующие имена полей. Вместо «offerUrl / deviceId / referrer» в коде идут `culinary`-кодовые имена: `spiceLantern`, `bazaarSaffron`, `copperPlatter`, `tableStory`, `dishCollection`. Зачем спрятали: усложнить поиск по строкам при модерации и автоматическом скане APK.

Плейсхолдер `{{OFFER_URL}}`. Если сервер не хочет светить полный URL в ответе, клиент подставляет зашитый `https://eternalsiegeapplication.space/`. Зачем: гибко менять лендинг на сервере, а в APK оставить базовый хост.

WebSocket вместо простого HTTP GET. Запрос идёт как Upgrade: websocket на `/api/anatoliaTable`. Зачем: меньше похоже на классический remote-config эндпоинт в статических сканерах; ответ приходит одним текстовым фреймом JSON.

Custom Tabs, не WebView. Оффер открывается во внешней вкладке браузера (`androidx.browser`). Зачем: не держать WebView в UI белой игры, но всё равно увести пользователя на лендинг. Класс WebView для показа страниц не используется — только `WebSettings.getDefaultUserAgent` для UA.

R8-обфускация и Pairip. Классы склеены в defpackage с короткими именами; Application обёрнут в com.pairip.application (Play Integrity / licensecheck). Зачем: затруднить ручной разбор и формально пройти проверки лицензии.

Роль подозрительных доменов

eternalsiegeapplication.space — центральный домен схемы. На шагах 4–6 с него по WebSocket приходит решение клоаки (это «мозг»). На шаге 6 при {{OFFER_URL}} тот же хост становится базой лендинга. На момент проверки VirusTotal: 0/91, корень отвечает {"error": "Not Found"} без редиректа (боевой путь — API /api/anatoliaTable и динамический tableStory, а не главная страница). Регистратор: NAMECHEAP INC, домен свежий.

Финал для пользователя

После всех проверок «боевой» пользователь оказывается во вкладке Custom Tabs на URL из ответа сервера (поле tableStory) либо на https://eternalsiegeapplication.space/, если пришёл плейсхолдер {{OFFER_URL}}. Точный тип финального сайта (казино, беттинг, виагра-оффер и т.п.) в клиенте не зашит — адрес динамический. На момент пайплайн-проверки корень домена отдавал только JSON Not Found, то есть публичная заглушка; реальный лендинг, скорее всего, отдаётся персонализированно через API. «Белый» пользователь просто остаётся в каталоге Anatolia / tic-tac-toe и оффер не видит.